

Exp No: 5 Date:	Dynamic programming – Knapsack problem, Subset sum problem, longest common subsequence problem
----------------------------------	---

AIM:

The aim of Dynamic Programming is to solve complex problems by breaking them into overlapping subproblems and storing their solutions to avoid redundant work.

PROCEDURE:

1. To maximize the total value of items included in a knapsack without exceeding its capacity, given weights and values of the items.
2. To determine if a subset of a given set of integers sums up to a specified value. To find the length of the longest subsequence common to two sequences, where the subsequence maintains the order of elements.

PROBLEM-1:

ITEM	WEIGHT	VALUE
1	1	1
2	3	4
3	4	5
4	5	7

Knapsack Capacity = 7

Code:

```
def
dp_example_1()
```

```

weights=[1,3,4,5
]
    values = [1, 4, 5, 7]
    capacity = 7

    n = len(values)

    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1,
            capacity + 1): if
            weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],

                        dp[i - 1][w]
                )
    else:

        dp[i][w] = dp[i - 1][w]

    print("Maximum value (Knapsack A):",

        dp[n][capacity])

dp_example_1()

```

Output:

Maximum value (Knapsack A): 9

RESULT :

Thus, the Branch and Bound technique was successfully executed and verified using Python